

Deep Learning & Intelligent Systems

Deep Learning Foundations

Niteesh K R

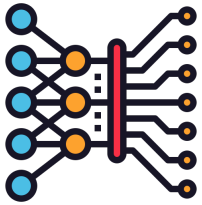
Computer Applications

Deep Learning & Intelligent Systems

Deep Learning Foundations

Deep Learning – Introduction, Neurons

Niteesh K R
Computer Applications

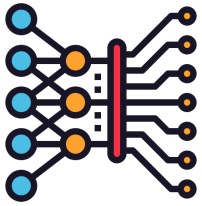


Deep Learning & Intelligent Systems

Deep Learning: Introduction and Neurons



- Deep Learning is a specialised branch of Machine Learning that learns useful representations directly from data using multiple computational layers
- It forms the foundation of modern intelligent systems for image understanding, speech recognition, language processing, recommendation, and automation
- This topic introduces:
  - What Deep Learning is and why it is important
  - How biological neurons inspired artificial neurons
- The focus is on building conceptual understanding before studying neural network architectures and learning algorithms



Deep Learning & Intelligent Systems

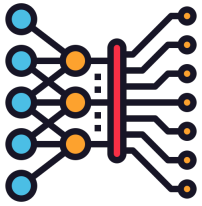
From Artificial Intelligence to Deep Learning

PES UNIVERSITY

- Artificial Intelligence (AI) aims to build systems capable of performing tasks that normally require human intelligence
- Machine Learning (ML) enables computers to learn patterns from data instead of relying entirely on explicitly programmed rules
- Deep Learning (DL) is a specialised branch of Machine Learning that uses multi-layer neural networks
- Relationship between the fields:
  - Artificial Intelligence $\supset$ Machine Learning $\supset$ Deep Learning
  - Each inner field is a specialised subset of the previous one
- Deep Learning is especially suitable for solving complex problems involving large and high-dimensional datasets

What is Deep Learning?

- Deep Learning is a data-driven approach in which a model automatically learns useful representations from training examples
- The term **deep** refers to the presence of multiple computational layers between the input and the output
- Lower layers usually learn simple patterns, while deeper layers combine them into increasingly complex representations
- Typical learning hierarchy:
  - Images: Edges $\rightarrow$ Shapes $\rightarrow$ Objects
  - Speech: Sound fragments $\rightarrow$ Phonemes $\rightarrow$ Words
  - Text: Tokens $\rightarrow$ Phrases $\rightarrow$ Meaning
- Unlike conventional approaches, these representations are learned automatically during training



Deep Learning & Intelligent Systems

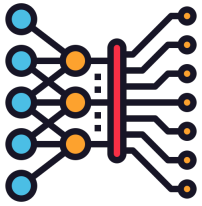
Representation Learning

PES UNIVERSITY

- A representation is the internal form used by a model to describe input data
- Better representations simplify the final prediction or classification task
- Deep Learning automatically discovers useful representations from data without manually designing features
- Example hierarchy for face recognition:
  - Edges and local intensity changes
  - Facial components such as eyes, nose and mouth
  - Identity-specific facial patterns
- These representations emerge naturally during training rather than being explicitly programmed

Why Deep Learning?

- Many real-world problems involve complex data that cannot be solved effectively using manually written rules
- Deep Learning can process different forms of data including images, text, speech, video and sensor signals
- A single model can simultaneously learn useful features and perform the final prediction task
- Practical success of Deep Learning became possible because of:
  - Large datasets
  - GPUs and specialised hardware
  - Improved learning algorithms
  - Frameworks such as TensorFlow and Keras
- Deep Learning should be selected when sufficient data and computational resources are available



Deep Learning & Intelligent Systems

Traditional Programming, ML, DL

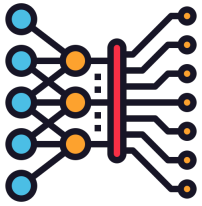
- Traditional Programming: Humans provide rules and data to produce the required output
- Machine Learning: Data and expected outputs are used to learn a predictive model
- Deep Learning: Neural networks automatically learn feature representations as well as prediction behaviour
- Comparison:
  - Traditional Programming: Rules + Data $\rightarrow$ Output
  - Machine Learning: Data + Labels $\rightarrow$ Model
  - Deep Learning: Data + Labels $\rightarrow$ Representations + Model
- Deep Learning reduces dependence on manually engineered features but still requires high-quality data

Why Deep Learning Became Practical

- Deep Learning concepts existed for decades, but widespread success became possible only after several technological advances
- Large datasets: Availability of labelled and unlabelled data for model training
- Powerful hardware: GPUs and specialised accelerators enabled efficient large-scale computation
- Improved algorithms: Better optimisation, initialisation and regularisation techniques improved training reliability
- Software ecosystem: Frameworks such as TensorFlow and Keras simplified model development and experimentation
- Cloud platforms such as Kaggle provide affordable access to GPU resources for learning and research

Common Applications of Deep Learning

- Computer Vision: Image classification, object detection, medical imaging and quality inspection
- Natural Language Processing: Machine translation, summarisation, question answering and conversational AI
- Speech and Audio: Speech recognition, speaker identification, voice assistants and audio classification
- Healthcare: Disease screening, physiological signal analysis and clinical decision support
- Intelligent Systems: Recommendation systems, anomaly detection, predictive maintenance and autonomous navigation
- The common principle across all applications is learning useful hierarchical representations directly from data

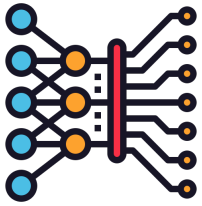


Deep Learning & Intelligent Systems

Limitations and Practical Challenges

PES UNIVERSITY

- Deep Learning models often require large quantities of suitable training data
- Training may demand significant computational resources, memory, time and energy
- Models can learn unwanted bias or noise present in the training dataset
- Complex neural networks are often difficult to interpret and explain
- Strong training performance does not necessarily guarantee good performance on unseen data
- Deep Learning should therefore be selected based on the problem requirements rather than treated as a universal solution



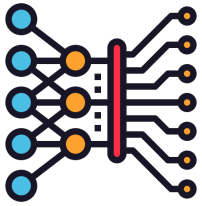
Deep Learning & Intelligent Systems

The Brain as an Inspiration

- Artificial Neural Networks are inspired by the idea that intelligent behaviour can emerge from many simple interconnected processing units
- In the nervous system, biological neurons receive signals, process them and communicate with other neurons
- Artificial neurons imitate this broad computational idea using numerical inputs, adjustable weights and an output
- The inspiration is conceptual rather than biological:
  - Biological neurons communicate using electrochemical signals
  - Artificial neurons perform mathematical computations
- Understanding this analogy helps explain the organisation of neural networks

Biological Neuron: Overview

- A biological neuron is a specialised cell that receives, processes and transmits information throughout the nervous system
- Communication between neurons occurs through electrical and chemical signals
- A simplified neuron consists of four major parts:
  - Dendrites
  - Cell body (Soma)
  - Axon
  - Synaptic terminals
- Each part contributes to receiving, processing or transmitting information
- Artificial neurons are inspired by this simplified model rather than the complete biological neuron



Deep Learning & Intelligent Systems

Dendrites, Soma and Axon

PES UNIVERSITY

- Dendrites receive signals from neighbouring neurons
- The cell body (soma) integrates the incoming signals before deciding the neuron's response
- The axon carries the generated electrical impulse away from the cell body
- Synaptic terminals at the end of the axon communicate with other neurons through synapses
- This receive $\rightarrow$ integrate $\rightarrow$ transmit process forms the basis for the artificial neuron model

Simplified Working of a Biological Neuron

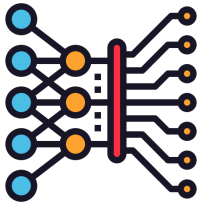
- Step 1: Dendrites receive signals from neighbouring neurons or sensory receptors
- Step 2: The cell body combines and processes the incoming signals
- Step 3: If the combined effect exceeds a threshold, the neuron generates an electrical impulse
- Step 4: The impulse travels through the axon and reaches the synaptic terminals
- Step 5: The signal is transmitted to other neurons through synapses
- This receive $\rightarrow$ combine $\rightarrow$ respond pattern inspires the design of artificial neurons

Artificial Neuron and its Components

- An artificial neuron is a mathematical processing unit that forms the basic building block of a neural network
- Major components include:
  - Inputs ($x_1, x_2, \ldots, x_n$)
  - Weights ($w_1, w_2, \ldots, w_n$)
  - Bias (b)
  - Weighted Sum
  - Output after an activation function
- During training, the weights and bias are adjusted to improve prediction accuracy
- Activation functions will be studied in detail later in Unit I

Biological and Artificial Neurons: Mapping

- The biological neuron provides an intuitive analogy for understanding an artificial neuron
- Useful mapping between the two systems:
  - Dendrites $\leftrightarrow$ Input values
  - Synaptic strength $\leftrightarrow$ Weights
  - Cell body $\leftrightarrow$ Weighted sum
  - Firing decision $\leftrightarrow$ Activation function
  - Axon $\leftrightarrow$ Output
- The mapping is conceptual and should not be interpreted as an exact biological simulation

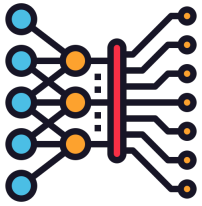


Deep Learning & Intelligent Systems

Biological Neurons vs. Artificial Neurons



- Biological neurons communicate using electrochemical signals, whereas artificial neurons perform numerical computations
- Biological learning modifies complex neural connections, while artificial learning updates weights and bias values
- Biological brains are highly energy efficient, whereas deep neural networks may require significant computational resources
- Artificial neurons are simplified computational abstractions designed for learning from data
- Although inspired by the brain, an artificial neural network is **not** a digital replica of the human brain

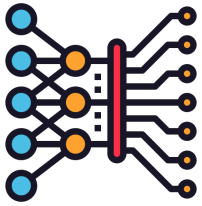


Deep Learning & Intelligent Systems

Illustrative Artificial Neuron Example

PES UNIVERSITY

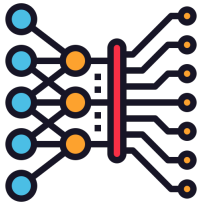
- Consider an artificial neuron designed to identify students requiring academic support
- Possible inputs include:
  - Attendance percentage
  - Assignment submission rate
  - Recent assessment performance
- Each input is assigned a weight according to its importance
- The neuron combines the weighted inputs with a bias to produce an output score
- This input $\rightarrow$ weight $\rightarrow$ combine $\rightarrow$ output flow forms the basis of all neural networks



Deep Learning & Intelligent Systems

Summary

- Deep Learning is a specialised branch of Machine Learning that learns hierarchical representations using multiple computational layers
- It is especially useful for complex data such as images, speech, text, video, and sensor signals
- Representation learning reduces dependence on manually engineered features
- Artificial neurons are simplified mathematical units inspired by the receive $\rightarrow$ combine $\rightarrow$ respond behaviour of biological neurons
- The biological analogy is useful for understanding neural network terminology, but artificial neurons are not exact replicas of biological neurons



Deep Learning & Intelligent Systems

Brainstorming Questions

- Why is representation learning useful for image, speech, and text data?
- Why are different inputs assigned different weights in an artificial neuron?
- In what ways is the biological neuron analogy useful?
- Where does the biological neuron analogy become misleading?
- When might a conventional Machine Learning model be preferable to a Deep Learning model?
- How does the receive $\rightarrow$ combine $\rightarrow$ respond pattern lead to the mathematical idea of a perceptron?

References

- Goodfellow, I., Bengio, Y., and Courville, A., *Deep Learning*, MIT Press, Chapter 1
- Krohn, J., Beyleveld, G., and Bassens, A., *Deep Learning Illustrated*, Pearson Education, 2020
- TensorFlow, *TensorFlow 2.16 API Documentation*
  - https://www.tensorflow.org/api_docs/python/tf/keras
- Keras, *Introduction to Keras for Engineers*
  - https://keras.io/getting_started/intro_to_keras_for_engineers/



Thank You

Niteesh K R

Assistant Professor

Department of Computer Applications

niteeshkr@pes.edu

080-26721983 Extn: 233



Deep Learning & Intelligent Systems

Deep Learning Foundations

Niteesh K R

Computer Applications

Deep Learning & Intelligent Systems

Deep Learning Foundations

MLPs and Feedforward Networks

Niteesh K R
Computer Applications

From a Single Neuron to a Neural Network

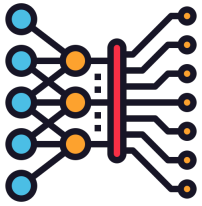
- An artificial neuron receives numerical inputs, combines them using weights and a bias, and produces an output
- A single neuron can represent only a limited relationship between the input features and the expected output
- Real-world problems often involve complex interactions that cannot be handled by one computational unit
- Neural networks address this limitation by connecting multiple artificial neurons into organised layers
  - Each neuron performs part of the overall computation
  - Outputs from one layer become inputs to the next
  - Successive layers learn increasingly useful representations

Why is One Artificial Neuron Not Enough?

- A single neuron combines all input features in one computational step
- It may be sufficient when the relationship between inputs and output is relatively simple
- Complex datasets contain interactions among several features
- One weighted combination cannot always represent these interactions effectively
- A single neuron does not create intermediate representations of complex feature combinations
- Multiple neurons arranged in layers increase the representational capability of the model

Complex Feature Interactions

- Real-world predictions frequently depend on combinations of several features
- Examples include:
  - Medical diagnosis: age, symptoms, test values, and their interactions
  - Fraud detection: amount, location, device, time, and transaction history
  - Image recognition: edges, textures, shapes, and object parts
- Multiple neurons divide a complex computation into smaller processing steps
- Each layer transforms the representation received from the preceding layer
- Network depth helps identify relationships that may not be directly visible in the raw input



Deep Learning & Intelligent Systems

Introduction to Multilayer Perceptrons

PES UNIVERSITY

- A Multilayer Perceptron (MLP) is a neural network composed of multiple layers of artificial neurons
- A typical MLP contains:
  - One input layer
  - One or more hidden layers
  - One output layer
- Neurons in one layer are usually connected to neurons in the next layer
- MLPs are commonly used for classification and regression involving structured numerical data
- They provide a foundation for understanding deeper neural network architectures

Basic Architecture of an MLP

- The architecture of an MLP can be represented as:

$$\text{Input Layer} \rightarrow \text{Hidden Layer(s)} \rightarrow \text{Output Layer}$$

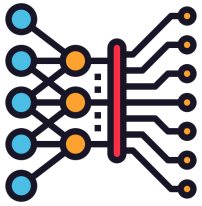
- The input layer receives the feature values supplied to the network
- The hidden layers transform the input through connected artificial neurons
- The output layer produces the prediction required by the application
- The number of hidden layers and neurons determines the size and capacity of the network

Input, Hidden and Output Layers

- Input Layer
  - Receives input features from the dataset
  - Acts as the entry point of the network
- Hidden Layer(s)
  - Perform computations using weights and biases
  - Learn intermediate representations of the data
- Output Layer
  - Produces the final prediction or decision
  - Its size depends on the learning task

How Information Moves Through an MLP

- Each layer receives information from the preceding layer and produces a new representation
- The input layer normally passes feature values into the network without learning parameters
- A hidden neuron combines the outputs of the previous layer using its own weights and bias
- Earlier hidden layers may identify simple feature combinations
- Deeper hidden layers combine them into more abstract patterns
- Processing continues layer by layer until the output is produced



Deep Learning & Intelligent Systems

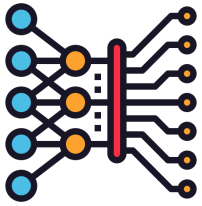
Hidden Layers and Hierarchical Learning

PES UNIVERSITY

- Hidden layers are called *hidden* because their intermediate outputs are not directly observed by the user
- Each hidden layer transforms the preceding representation into a new numerical representation
- Earlier layers generally identify simpler relationships
- Deeper layers combine simpler relationships into more abstract concepts
- Multiple hidden layers therefore support hierarchical representation learning
- Greater depth can improve capacity, but also increases computation and data requirements

Layered Representation: Student Performance

- Consider an MLP designed to predict whether a student requires academic intervention
- Input features may include:
  - Attendance and assignment performance
  - Test and laboratory scores
  - Previous CGPA
- An early hidden layer may identify attendance and assessment patterns
- A later hidden layer may combine these patterns into an indicator of academic risk
- The output layer may classify the student as low-risk, medium-risk, or high-risk



Deep Learning & Intelligent Systems

Output Layer and Network Capacity

PES UNIVERSITY

- The number of output neurons depends on the prediction task
  - Regression: commonly one output neuron
  - Binary classification: commonly one output neuron
  - Multiclass classification: commonly one neuron per category
- Increasing the number of hidden neurons increases the width of the network
- Increasing hidden layers increases the depth of the network
- Larger networks contain more trainable parameters and greater learning capacity
- Excess capacity may increase computation and the risk of overfitting

Weights, Biases and the Weighted Sum

- Every connection entering a neuron has an associated numerical value called a weight
- A weight controls the influence of an input on the neuron
- Each neuron normally has an additional trainable parameter called the bias
- For inputs $x_1, x_2, \ldots, x_n$, the weighted sum is:

$$z = \sum_{i=1}^{n} w_i x_i + b$$

- During training, weights and biases are adjusted to improve the predictions of the network
- The transformation applied to z will be studied under activation functions

In-Class Problem 1: Weighted Sum

- Consider a neuron with:

$$x_1 = 2, \quad x_2 = 3, \quad x_3 = 1$$

$$w_1 = 0.5, \quad w_2 = -1, \quad w_3 = 2, \quad b = 1$$

- Calculate the weighted sum:

$$z = w_1x_1 + w_2x_2 + w_3x_3 + b$$

In-Class Problem 1: Weighted Sum

- Solution:

$$z = (0.5 \times 2) + (-1 \times 3) + (2 \times 1) + 1$$

$$z = 1 - 3 + 2 + 1 = \boxed{1}$$

- The value z becomes the input to an activation function

Dense or Fully Connected Layers

- The most common computational layer in an MLP is the Dense Layer
- A Dense Layer is also called a Fully Connected Layer
- Every neuron in the previous layer is connected to every neuron in the Dense Layer
- For n_{in} input values and n_{out} neurons:

$$\text{Number of weights} = n_{\text{in}} \times n_{\text{out}}$$

- When each output neuron has one bias:

$$\text{Trainable parameters} = (n_{\text{in}} + 1)n_{\text{out}}$$

In-Class Problem 2: Dense-Layer Parameters

- A Dense Layer receives 5 input values and contains 3 neurons
- Number of weights:

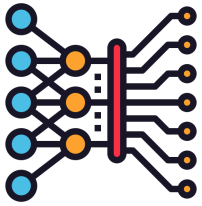
$$5 \times 3 = 15$$

- Number of biases: 3
- Total trainable parameters:

$$(5 + 1) \times 3 = \boxed{18}$$

- A second Dense Layer receives these 3 outputs and contains 2 neurons
- Its parameter count is:

$$(3 + 1) \times 2 = \boxed{8}$$

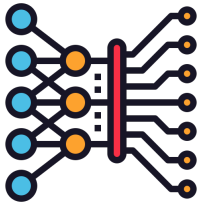


Deep Learning & Intelligent Systems

Introducing MLPs with TensorFlow

PES UNIVERSITY

- TensorFlow provides the Keras API for constructing neural networks
- The minimum APIs required at this stage are:
  - `tf.keras.Sequential`
  - `tf.keras.Input`
  - `tf.keras.layers.Dense`
  - `model.summary()`
- `Sequential` arranges layers in a linear order
- `Dense` creates a fully connected layer
- Training APIs such as `compile()` and `fit()` will be introduced after loss functions and optimisation



Deep Learning & Intelligent Systems

Building a Small MLP in TensorFlow

PES UNIVERSITY

- Create a simple network for a dataset containing five input features:

```
model = tf.keras.Sequential([
    tf.keras.Input(shape=(5,)),
    tf.keras.layers.Dense(8, activation="relu"),
    tf.keras.layers.Dense(4, activation="relu"),
    tf.keras.layers.Dense(1)
])
model.summary()
```

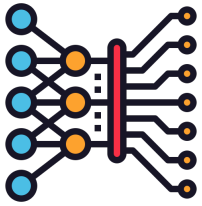
- The model contains two hidden Dense Layers followed by one output neuron
- The activation argument is shown here only to complete the architecture; activation functions are covered next
- `model.summary()` displays layer shapes and parameter counts

Feedforward Neural Networks

- A Feedforward Neural Network (FFNN) processes information in one direction
- The computational path is:

$$\text{Input} \rightarrow \text{Hidden Layer(s)} \rightarrow \text{Output}$$

- Every layer processes the output generated by the preceding layer
- There are no feedback loops or cyclic connections
- During prediction, information does not return to an earlier layer
- A traditional MLP is a type of Feedforward Neural Network



Deep Learning & Intelligent Systems

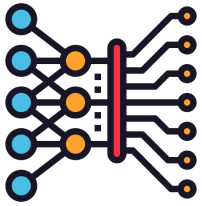
MLP, Perceptron and FFNN

PES UNIVERSITY

- A Single-Layer Perceptron contains one computational layer and no hidden layers
- A Multilayer Perceptron contains one or more hidden layers of fully connected neurons
- A Feedforward Neural Network is defined by the one-directional flow of information
- Therefore:
  - Every traditional MLP is feedforward
  - Feedforward Neural Network is the broader category
  - An MLP is a specific fully connected feedforward architecture
- Hidden layers give an MLP greater representational capacity than a single-layer perceptron

Applications, Advantages and Limitations

- MLP applications include credit-risk assessment, customer churn, medical diagnosis, student-performance prediction, and numerical forecasting
- Advantages:
  - Support classification and regression
  - Learn complex relationships among features
  - Create intermediate feature representations
- Limitations:
  - Dense connectivity may produce many parameters
  - Large MLPs require more data and computation
  - They may overfit when training data is limited
- Specialised architectures are often preferred for images and sequential data



Deep Learning & Intelligent Systems

Summary

- An MLP connects artificial neurons across input, hidden, and output layers
- Hidden layers learn intermediate representations and increase model capacity
- Each neuron computes a weighted sum using its inputs, weights, and bias
- Dense Layers connect every input value to every neuron in the next layer

Summary

- Dense-Layer parameters can be calculated using:

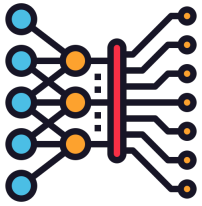
$$(n_{\text{in}} + 1)n_{\text{out}}$$

- An MLP follows the feedforward principle, with information moving from input to output
- Activation functions determine how weighted sums are transformed in the next topic

Deep Learning & Intelligent Systems

Brainstorming Questions

- Why is a single artificial neuron insufficient for many real-world problems?
- How do hidden layers improve the representational capability of an MLP?
- What is the difference between increasing the depth and increasing the width of a network?
- Why can the number of parameters grow rapidly in Dense Layers?
- Is every Feedforward Neural Network an MLP? Justify your answer
- Why might a specialised architecture be preferred over an MLP for images or sequential data?



Deep Learning & Intelligent Systems

References

- Goodfellow, I., Bengio, Y., and Courville, A., *Deep Learning*, MIT Press, Chapter 6
- Géron, A., *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, O'Reilly Media, 3rd Edition, 2022
- TensorFlow 2.16, *tf.keras.Sequential*
  - https://www.tensorflow.org/api_docs/python/tf/keras/Sequential
- TensorFlow 2.16, *tf.keras.layers.Dense*
  - https://www.tensorflow.org/api_docs/python/tf/keras/layers/Dense
- TensorFlow, *The Sequential Model*
  - https://www.tensorflow.org/guide/keras/sequential_model



Thank You

Niteesh K R

Assistant Professor

Department of Computer Applications

niteeshkr@pes.edu

080-26721983 Extn: 233



Deep Learning & Intelligent Systems

Deep Learning Foundations

Niteesh K R

Computer Applications

Deep Learning & Intelligent Systems

Deep Learning Foundations

Activation Functions

Niteesh K R
Computer Applications

Why Activation Functions?

- An artificial neuron first calculates a weighted sum of inputs and bias
- The weighted sum is:

$$z = w_1x_1 + w_2x_2 + \cdots + w_nx_n + b$$

- By itself, this weighted sum is only a linear transformation
- An activation function converts z into the neuron's output
- Activation functions allow neural networks to learn non-linear patterns in data

Activation in an Artificial Neuron

- A neuron performs two main computation stages
- Stage 1: calculate the weighted sum

$$z = \sum_{i=1}^{n} w_i x_i + b$$

- Stage 2: apply the activation function

$$y = f(z)$$

- f represents the selected activation function
- Different activation functions transform the same z in different ways

Why Non-Linearity Matters

- A Dense layer without activation performs a linear transformation
- Multiple linear layers still collapse into one linear transformation
- Therefore, depth alone does not guarantee complex learning
- Non-linear activation functions help networks represent:
  - Curved decision boundaries
  - Threshold effects in real-world data
  - Interactions among multiple input features
- This is why hidden layers usually need non-linear activations

Common Activation Functions

- This lecture focuses on three syllabus-required activation functions
- ReLU returns zero for negative inputs and preserves positive inputs
- Sigmoid maps inputs into the range $(0, 1)$
- Tanh maps inputs into the range $(-1, 1)$
- Their output ranges influence how signals move through the network
- Hidden layers and output layers may require different activations

Rectified Linear Unit (ReLU)

- The Rectified Linear Unit is commonly called ReLU
- It is defined as:

$$f(x) = \max(0, x)$$

- If $x < 0$, ReLU returns 0
- If $x \geq 0$, ReLU returns x
- ReLU is computationally simple and widely used in hidden layers
- A limitation is that some neurons may keep outputting zero during training

ReLU Numerical Problem

- Compute ReLU for the following inputs:

  $$x = [-3,\ -1,\ 0,\ 2,\ 5]$$

- Apply the rule $f(x) = \max(0, x)$
- Solution:

  $$\text{ReLU}(-3) = 0, \quad \text{ReLU}(-1) = 0, \quad \text{ReLU}(0) = 0$$
  $$\text{ReLU}(2) = 2, \quad \text{ReLU}(5) = 5$$

- Final output:

  $$[0,\ 0,\ 0,\ 2,\ 5]$$

Sigmoid Activation Function

- The sigmoid function converts any real number into a value between 0 and 1
- It is defined as:

$$f(x) = \frac{1}{1+e^{-x}}$$

- Large negative inputs produce values close to 0
- $x = 0$ produces 0.5
- Large positive inputs produce values close to 1
- Sigmoid is commonly used for binary-classification output neurons

Sigmoid Numerical Problem

- Compute sigmoid for $x = 0$ and $x = 2$
- Formula:

$$\sigma(x) = \frac{1}{1+e^{-x}}$$

- For $x = 0$:

$$\sigma(0) = \frac{1}{1+e^{0}} = \frac{1}{2} = 0.5$$

- For $x = 2$:

$$\sigma(2) = \frac{1}{1+e^{-2}} \approx 0.881$$

- Larger positive inputs give stronger positive confidence

Hyperbolic Tangent (Tanh)

- Tanh maps real-valued inputs into the range $(-1, 1)$
- It is defined as:

$$f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

- Large negative inputs produce values close to -1
- $x = 0$ produces 0
- Large positive inputs produce values close to 1
- Tanh is zero-centred, unlike sigmoid

Tanh Numerical Problem

- Compute tanh for the following inputs:

  $x = [-1,\ 0,\ 1]$

- Using standard approximate values:

  $\tanh(-1) \approx -0.762$

  $\tanh(0) = 0$

  $\tanh(1) \approx 0.762$

- Final output:

  $[-0.762,\ 0,\ 0.762]$

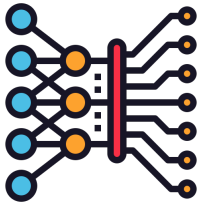
- Tanh preserves the sign of the input while compressing its magnitude

Comparing ReLU, Sigmoid and Tanh

- ReLU:
  - Output range: $[0, \infty)$
  - Common use: hidden layers
- Sigmoid:
  - Output range: $(0, 1)$
  - Common use: binary-classification output
- Tanh:
  - Output range: $(-1, 1)$
  - Common use: signed intermediate representations

Choosing an Activation Function

- Use ReLU as a practical default for many hidden layers
- Use sigmoid when one output neuron should represent a binary probability-like score
- Use tanh when negative and positive internal values are useful
- The activation must match the meaning expected from the layer output
- Hidden-layer activation controls feature transformation
- Output-layer activation controls prediction interpretation

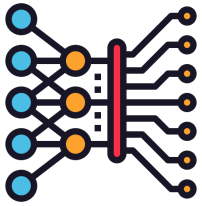


Deep Learning & Intelligent Systems

TensorFlow Activation APIs

PES UNIVERSITY

- TensorFlow provides activation functions through `tf.keras.activations`
- Function: `tf.keras.activations.relu(x)`
- Function: `tf.keras.activations.sigmoid(x)`
- Function: `tf.keras.activations.tanh(x)`
- Parameter:
  - `x`: input tensor to be transformed
- Return Type:
  - Tensor containing transformed values



Deep Learning & Intelligent Systems

Applying Activations to a Tensor

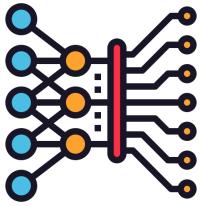
PES UNIVERSITY

- Apply all three activation functions to the same tensor

```
import tensorflow as tf
x = tf.constant([-2.0, -1.0, 0.0, 1.0, 2.0])

print(tf.keras.activations.relu(x).numpy())
print(tf.keras.activations.sigmoid(x).numpy())
print(tf.keras.activations.tanh(x).numpy())
```

- Expected pattern:
  - ReLU removes negative values
  - Sigmoid compresses values into $(0, 1)$
  - Tanh preserves sign and compresses values into $(-1, 1)$



Deep Learning & Intelligent Systems

Manual Artificial Neuron

- This example performs the weighted-sum and activation stages manually

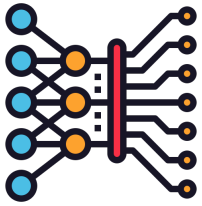
```
import tensorflow as tf
x = tf.constant([2.0, 3.0])
w = tf.constant([0.5, -0.2])
b = tf.constant(0.1)

z = tf.reduce_sum(x * w) + b
output = tf.keras.activations.relu(z)
```

- Manual calculation:

$$z = (2.0 \times 0.5) + (3.0 \times -0.2) + 0.1 = 0.5$$

$$\text{ReLU}(0.5) = 0.5$$



Deep Learning & Intelligent Systems

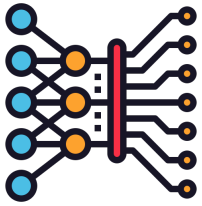
Dense Layer with Activation

PES UNIVERSITY

- TensorFlow allows activation to be specified while creating a Dense layer

```
layer = tf.keras.layers.Dense(
    units=8,
    activation="relu")
```

- `units=8` creates eight neurons
- `activation="relu"` applies ReLU to each neuron output
- Internally, the layer first calculates weighted sums
- Then it applies the activation function before passing values forward



Deep Learning & Intelligent Systems

Feedforward Network Example

- Build a small feedforward network with hidden and output activations

```
import tensorflow as tf

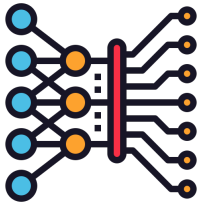
model = tf.keras.Sequential([
    tf.keras.Input(shape=(4,)),
    tf.keras.layers.Dense(8, activation="relu"),
    tf.keras.layers.Dense(4, activation="tanh"),
    tf.keras.layers.Dense(1, activation="sigmoid")
])

model.summary()
```

- This lecture demonstrates architecture and activation flow, not training

Best Practices

- Do not stack Dense layers without activation unless there is a clear reason
- Prefer ReLU for basic feedforward hidden layers
- Avoid using sigmoid repeatedly in many hidden layers by default
- Match the output activation with the prediction task
- Check output ranges when debugging model behaviour
- Compare activation choices experimentally after the model is trainable



Deep Learning & Intelligent Systems

Summary

- Activation functions convert weighted sums into neuron outputs
- Non-linear activations allow multilayer networks to learn complex patterns
- ReLU is simple and widely used in hidden layers
- Sigmoid is useful for binary-classification output
- Tanh is zero-centred and preserves signed information
- In the next lecture, these outputs will be compared with target values using loss functions

Deep Learning & Intelligent Systems

Brainstorming Questions

- Why does a multilayer network without activations behave like a linear model?
- Why might ReLU train faster than sigmoid in hidden layers?
- When can sigmoid be a good output activation but a poor hidden-layer default?
- How does the output range of tanh differ from sigmoid?
- If a neuron always outputs zero after ReLU, what problem may occur during learning?
- How should the output activation change for binary and multi-class classification?

References

- Jon Krohn, Grant Beyleveld and Aglae Bassens, *Deep Learning Illustrated*, Pearson Education – Chapters 4 and 7
- Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press – Chapter 6
- TensorFlow 2.16 – `tf.keras.activations.relu`
- TensorFlow 2.16 – `tf.keras.activations.sigmoid`
- TensorFlow 2.16 – `tf.keras.activations.tanh`
- TensorFlow 2.16 – `tf.keras.layers.Dense`
- TensorFlow 2.16 – `tf.keras.Sequential`



Thank You

Niteesh K R

Assistant Professor

Department of Computer Applications

niteeshkr@pes.edu

080-26721983 Extn: 233



Deep Learning & Intelligent Systems

Deep Learning Foundations

Niteesh K R

Computer Applications

Deep Learning & Intelligent Systems

Deep Learning Foundations

Loss Functions

Niteesh K R
Computer Applications

Why Loss Functions are Needed

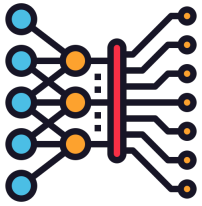
- A neural network produces a prediction for each input
- During training, the prediction must be compared with the expected output
- A loss function measures the difference between prediction and ground truth
- Smaller loss means the prediction is closer to the expected value
- Larger loss means the prediction contains greater error
- Training tries to reduce loss by adjusting weights and biases

Loss in the Training Process

- The model first performs a feedforward computation
- It produces a predicted output $\hat{y}$
- The loss function receives:
  - Actual output: y
  - Predicted output: $\hat{y}$
- It converts prediction error into a single numerical value
- This value guides later parameter updates during training
- Low training loss alone does not guarantee good generalisation

Loss Function versus Evaluation Metric

- A loss function guides learning during model training
- An evaluation metric helps humans interpret model performance
- Loss functions are selected for mathematical suitability during optimisation
- Metrics are selected based on practical meaning
- Example for regression:
  - Loss: Mean Squared Error
  - Metric: Mean Absolute Error
- Example for classification:
  - Loss: Cross-Entropy
  - Metric: Accuracy



Deep Learning & Intelligent Systems

Choosing a Loss Function

PES UNIVERSITY

- The loss function must match the prediction problem
- Use Mean Squared Error for continuous numerical outputs
- Use Binary Cross-Entropy for two-class classification
- Use Categorical Cross-Entropy for multiclass classification with one-hot labels
- Use Sparse Categorical Cross-Entropy for multiclass classification with integer labels
- The output layer and loss function must be compatible

Mean Squared Error

- Mean Squared Error, abbreviated as MSE, measures average squared prediction error
- It is commonly used for regression problems
- Formula:

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

- y_i represents the actual value
- $\hat{y}_i$ represents the predicted value
- A perfect prediction produces an MSE of 0

MSE Numerical Problem

- Actual values:

  $$y = [10,\ 20,\ 30]$$

- Predicted values:

  $$\hat{y} = [12,\ 18,\ 29]$$

- Errors:

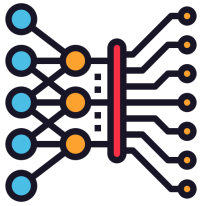
  $$10 - 12 = -2, \quad 20 - 18 = 2, \quad 30 - 29 = 1$$

- Squared errors:

  $$[4,\ 4,\ 1]$$

- Therefore:

  $$MSE = \frac{4+4+1}{3} = 3$$



Deep Learning & Intelligent Systems

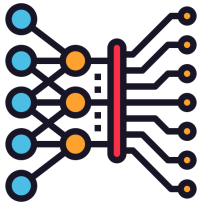
TensorFlow Mean Squared Error API

PES UNIVERSITY

- TensorFlow provides MSE through

  `tf.keras.losses.MeanSquaredError`
- Function:

  `tf.keras.losses.MeanSquaredError()`
- Inputs:
  - `y_true`: actual target values
  - `y_pred`: predicted values
- Return Type:
  - Tensor containing the computed loss
- Default behaviour returns the average loss across observations



Deep Learning & Intelligent Systems

Computing MSE in TensorFlow

PES UNIVERSITY

- The following program computes Mean Squared Error

```
import tensorflow as tf

y_true = tf.constant([[10.0], [20.0], [30.0]])
y_pred = tf.constant([[12.0], [18.0], [29.0]])

mse = tf.keras.losses.MeanSquaredError()
loss = mse(y_true, y_pred)
print(loss.numpy())
```

- Expected output:

```
3.0
```

Cross-Entropy Loss

- Cross-Entropy is used for classification problems
- It compares actual class labels with predicted probabilities
- The model is penalised when it assigns low probability to the correct class
- Confident incorrect predictions receive a large penalty
- Correct predictions with high confidence receive a small penalty
- Its exact form depends on whether the task is binary or multiclass

Binary Cross-Entropy

- Binary Cross-Entropy is used when there are two possible classes
- The actual label is represented as 0 or 1
- The model usually produces one sigmoid probability
- Formula:

$$L = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

- y represents the actual binary label
- $\hat{y}$ represents the predicted probability

BCE Numerical Problem

- Suppose the actual label is:

  $$y = 1$$

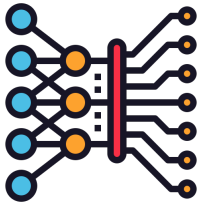
- The model predicts:

  $$\hat{y} = 0.9$$

- Binary Cross-Entropy:

  $$L = -[1 \log(0.9) + (1 - 1) \log(1 - 0.9)]$$
  $$L = -\log(0.9)$$
  $$L \approx 0.105$$

- The loss is small because the model assigns high probability to the correct class



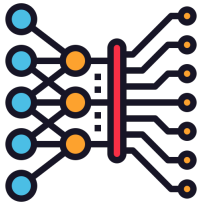
Deep Learning & Intelligent Systems

TensorFlow Binary Cross-Entropy API

PES UNIVERSITY

- TensorFlow provides BCE through `tf.keras.losses.BinaryCrossentropy`
- Function:

  `tf.keras.losses.BinaryCrossentropy()`
- Inputs:
  - `y_true`: actual binary labels
  - `y_pred`: predicted probabilities or logits
- Important parameter:
  - `from_logits`: set to `True` when predictions are raw logits



Deep Learning & Intelligent Systems

Computing BCE in TensorFlow

PES UNIVERSITY

- The following program computes Binary Cross-Entropy from probabilities

```
import tensorflow as tf

y_true = tf.constant([[1.0], [0.0], [1.0]])
y_pred = tf.constant([[0.9], [0.2], [0.7]])

bce = tf.keras.losses.BinaryCrossentropy()
loss = bce(y_true, y_pred)
print(loss.numpy())
```

- A lower value means the predicted probabilities agree better with the labels

Categorical Cross-Entropy

- Categorical Cross-Entropy is used for multiclass classification
- The actual class is commonly represented using one-hot encoding
- The output layer usually contains one neuron per class
- The model produces a probability distribution across classes
- Formula:

$$L = -\sum_{i=1}^{C} y_i \log(\hat{y}_i)$$

- The loss focuses on the probability assigned to the correct class

One-Hot and Sparse Labels

- Consider three classes: Cat, Dog, and Bird
- If the actual class is Dog, its one-hot label is:

  $[0, 1, 0]$

- Sparse integer labels represent the same classes as:
  - 0 for Cat
  - 1 for Dog
  - 2 for Bird
- One-hot labels use `CategoricalCrossentropy`
- Integer labels use `SparseCategoricalCrossentropy`

CCE Numerical Problem

- Actual one-hot label:

  $$y = [0,\ 1,\ 0]$$

- Predicted probabilities:

  $$\hat{y} = [0.1,\ 0.8,\ 0.1]$$

- Categorical Cross-Entropy:

  $$L = -[0 \log(0.1) + 1 \log(0.8) + 0 \log(0.1)]$$
  $$L = -\log(0.8)$$
  $$L \approx 0.223$$

- Increasing the correct-class probability decreases the loss

TensorFlow Categorical Loss APIs

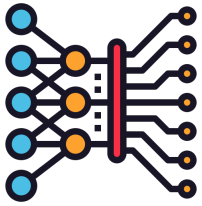
- Function for one-hot labels:

  `tf.keras.losses.CategoricalCrossentropy()`

- Function for integer labels:

  `tf.keras.losses.SparseCategoricalCrossentropy()`

- Inputs:
  - `y_true`: one-hot or integer labels
  - `y_pred`: class probabilities or logits
- Use `from_logits=True` when model outputs are raw logits



Deep Learning & Intelligent Systems

Computing CCE in TensorFlow

PES UNIVERSITY

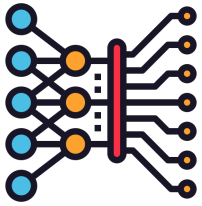
- The following program uses one-hot encoded labels

```
import tensorflow as tf

y_true = tf.constant([[0.0, 1.0, 0.0]])
y_pred = tf.constant([[0.1, 0.8, 0.1]])

cce = tf.keras.losses.CategoricalCrossentropy()
loss = cce(y_true, y_pred)
print(loss.numpy())
```

- The loss is small because the correct class receives probability 0.8



Deep Learning & Intelligent Systems

Using Loss in a Keras Model

PES UNIVERSITY

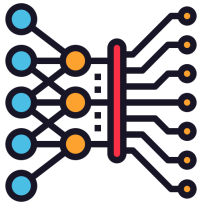
- A loss function is specified while compiling a Keras model

```
model.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"]
)
```

- The optimiser attempts to minimise the selected loss
- Metrics such as accuracy are reported separately
- The loss must match the problem type and output layer

Matching Output Layer and Loss

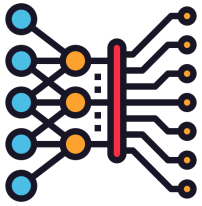
- Regression:
  - Output: one linear neuron
  - Loss: `MeanSquaredError`
- Binary classification:
  - Output: one sigmoid neuron
  - Loss: `BinaryCrossentropy`
- Multiclass classification with one-hot labels:
  - Output: softmax probabilities
  - Loss: `CategoricalCrossentropy`
- Multiclass classification with integer labels:
  - Loss: `SparseCategoricalCrossentropy`



Deep Learning & Intelligent Systems

Summary

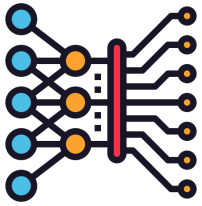
- Loss functions measure how wrong model predictions are
- MSE is mainly used for regression
- Binary Cross-Entropy is used for two-class classification
- Categorical Cross-Entropy is used for multiclass classification with one-hot labels
- Sparse Categorical Cross-Entropy is used for multiclass classification with integer labels
- In the next lecture, we study how loss is propagated backward to update weights



Deep Learning & Intelligent Systems

Brainstorming Questions

- Why does MSE penalise larger regression errors more strongly?
- Why is Cross-Entropy more suitable than MSE for probability-based classification?
- What can go wrong if the output activation and loss function do not match?
- When should Sparse Categorical Cross-Entropy be preferred over Categorical Cross-Entropy?
- Why can accuracy remain unchanged while loss still improves?
- Why should `from_logits=True` be used carefully?



Deep Learning & Intelligent Systems

References

- Jon Krohn, Grant Beyleveld and Aglae Bassens, *Deep Learning Illustrated*, Pearson Education – Chapters 4 and 7
- Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press – Chapters 5 and 6
- TensorFlow 2.16 – `tf.keras.losses.MeanSquaredError`
- TensorFlow 2.16 – `tf.keras.losses.BinaryCrossentropy`
- TensorFlow 2.16 – `tf.keras.losses.CategoricalCrossentropy`
- TensorFlow 2.16 – `tf.keras.losses.SparseCategoricalCrossentropy`
- TensorFlow Keras Guide – Training and evaluation with built-in methods



Thank You

Niteesh K R

Assistant Professor

Department of Computer Applications

niteeshkr@pes.edu

080-26721983 Extn: 233



Deep Learning & Intelligent Systems

Deep Learning Foundations

Niteesh K R

Computer Applications

Deep Learning & Intelligent Systems

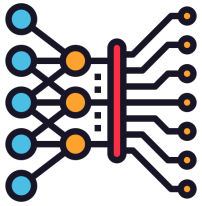
Deep Learning Foundations

Forward and Backpropagation

Niteesh K R
Computer Applications

How a Neural Network Learns

- A neural network begins with weights and biases that have not yet learned the required relationship
- It receives input features and produces a prediction
- The prediction is compared with the expected output using a loss function
- The model estimates how its trainable parameters contributed to the loss
- Weights and biases are adjusted to reduce future prediction error
- This learning process has two major phases: forward propagation and backpropagation



Deep Learning & Intelligent Systems

Forward and Backward Passes

PES UNIVERSITY

- Forward propagation moves information from the input layer to the output layer
- It calculates the prediction produced by the current weights and biases
- The loss function measures the difference between prediction and ground truth
- Backpropagation moves error-related information backward through the network
- It computes gradients for trainable parameters
- Training repeatedly alternates between forward and backward passes

Forward Propagation

- Forward propagation computes a network output from a given input
- Input features enter through the input layer
- Each layer calculates weighted sums using current weights and biases
- Activation functions transform the weighted sums
- The transformed values become inputs to the next layer
- The final layer produces the model prediction

Forward Propagation through One Neuron

- Consider a neuron receiving inputs x_1 and x_2
- The neuron first calculates the weighted sum:

$$z = w_1 x_1 + w_2 x_2 + b$$

- An activation function is then applied:

$$a = f(z)$$

- z is the value before activation
- a is the output passed to the next layer

Forward Propagation through Layers

- A multilayer network applies the same idea repeatedly
- Example flow:

  $$\text{Input} \rightarrow \text{Hidden Layer 1} \rightarrow \text{Hidden Layer 2} \rightarrow \text{Output}$$

- Hidden Layer 1 transforms the original input features
- Hidden Layer 2 combines the representations from Hidden Layer 1
- The output layer converts the final representation into a prediction
- The prediction depends on all current weights and biases

Forward Propagation Numerical Problem

- Consider one neuron with:
  - Inputs: $x_1 = 2,\ x_2 = 3$
  - Weights: $w_1 = 0.5,\ w_2 = -0.2$
  - Bias: $b = 0.1$
- Weighted sum:

$$z = (2 \times 0.5) + (3 \times -0.2) + 0.1$$

$$z = 1.0 - 0.6 + 0.1 = 0.5$$

- Using ReLU:

$$a = \max(0, 0.5) = 0.5$$

Manual Forward Propagation in TensorFlow

- The same neuron computation can be performed manually in TensorFlow

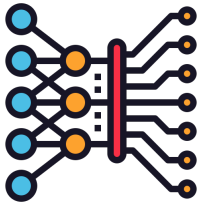
```
import tensorflow as tf

x = tf.constant([2.0, 3.0])
w = tf.constant([0.5, -0.2])
b = tf.constant(0.1)

z = tf.reduce_sum(x * w) + b
output = tf.keras.activations.relu(z)
print(z.numpy(), output.numpy())
```

- Expected output:

```
0.5 0.5
```



Deep Learning & Intelligent Systems

Forward Pass through a Keras Model

- A Keras model performs layer-by-layer forward propagation automatically

```
import tensorflow as tf

model = tf.keras.Sequential([
    tf.keras.Input(shape=(2,)),
    tf.keras.layers.Dense(4, activation="relu"),
    tf.keras.layers.Dense(1, activation="sigmoid")
])

x = tf.constant([[0.5, -1.0]])
prediction = model(x)
```

Interpreting the Forward Pass

- The input sample contains two numerical features
- The hidden layer contains four neurons using ReLU
- Each hidden neuron calculates a weighted sum and activation
- The output layer contains one neuron using sigmoid
- The sigmoid output lies between 0 and 1
- The prediction is not meaningful until the model has learned suitable parameters

From Prediction to Loss

- Forward propagation produces the model prediction
- The prediction is compared with the actual target
- A loss function converts the prediction error into a numerical value
- Example for binary classification:
  - Actual label: 1
  - Predicted probability: 0.30
  - The prediction receives a relatively high loss
- The network must identify which parameters contributed to this error

What is Backpropagation?

- Backpropagation computes gradients in a neural network
- It begins after the forward pass and loss calculation
- Error-related information moves from the output layer toward earlier layers
- The algorithm determines how each trainable parameter influenced the loss
- It uses the chain rule internally
- The calculated gradients are later used to update weights and biases

Essential Intuition behind Gradients

- A gradient describes how the loss changes when a parameter changes slightly
- A positive gradient means increasing the parameter may increase the loss
- A negative gradient means increasing the parameter may decrease the loss
- A large gradient magnitude means the loss is highly sensitive to that parameter
- A gradient close to zero means weak sensitivity
- To reduce loss, training usually moves opposite to the gradient direction

Backpropagation Essential Steps

- Step 1: Perform a forward pass using current parameters
- Step 2: Calculate the prediction and loss
- Step 3: Start from the output side of the network
- Step 4: Compute gradients for output-layer parameters
- Step 5: Propagate gradient information toward earlier layers
- Step 6: Use gradients to update trainable weights and biases

Why Backpropagation Moves Backwards

- Output error is known only after the prediction is produced
- Earlier layers influence the output indirectly through later layers
- Backpropagation begins at the loss and works toward the input side
- Each layer receives information about its contribution to the final error
- The gradient for a parameter depends on computations that happened after it
- The chain rule connects these dependent computations efficiently

Chain Rule Minimal Intuition

- Neural network computations are organised as dependent operations
- Example sequence:

$$\text{Weight} \rightarrow \text{Weighted Sum} \rightarrow \text{Activation} \rightarrow \text{Prediction} \rightarrow \text{Loss}$$

- Changing a weight affects the weighted sum
- The weighted sum affects the activation
- The activation affects the prediction and loss
- The chain rule combines these local effects

Simple Gradient Numerical Problem

- Consider the simple function:

$$y = x^2$$

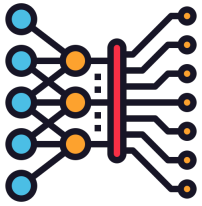
- Its gradient with respect to x is:

$$\frac{dy}{dx} = 2x$$

- At $x = 3$:

$$\frac{dy}{dx} = 2 \times 3 = 6$$

- Interpretation:
  - Around $x = 3$, increasing x increases y
  - The positive gradient shows the direction of increase



Deep Learning & Intelligent Systems

Automatic Differentiation in TensorFlow

PES UNIVERSITY

- TensorFlow provides automatic differentiation through `tf.GradientTape`
- A gradient tape records operations executed inside its context
- TensorFlow automatically watches trainable `tf.Variable` objects
- Function:

```
tf.GradientTape()
```

- Gradient calculation:

```
tape.gradient(target, sources)
```

- The returned gradients correspond to the supplied variables

Computing a Simple Gradient

- The following example computes the gradient of $y = x^2$

```
import tensorflow as tf

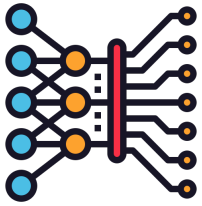
x = tf.Variable(3.0)

with tf.GradientTape() as tape:
    y = x ** 2

gradient = tape.gradient(y, x)
print(gradient.numpy())
```

- Expected output:

```
6.0
```



Deep Learning & Intelligent Systems

Forward Pass and Gradient Calculation

PES UNIVERSITY

- This code calculates loss and parameter gradients

```
x = tf.constant([[1.0, 2.0]])
y_true = tf.constant([[1.0]])

with tf.GradientTape() as tape:
    y_pred = model(x, training=True)
    loss = tf.reduce_mean((y_true - y_pred) ** 2)

gradients = tape.gradient(
    loss, model.trainable_variables)
```

- Gradients are calculated but not yet applied in this example

Understanding the GradientTape Example

- `model(x, training=True)` performs the forward pass
- `y_pred` stores the model prediction
- The example computes Mean Squared Error manually
- The tape records operations involved in producing the loss
- `model.trainable_variables` contains the model weights and biases
- `tape.gradient()` calculates one gradient for each trainable variable

One Simple Parameter Update

- A parameter can be moved opposite to its gradient
- A small learning rate controls the update size
- Basic update rule:

$$w_{\text{new}} = w_{\text{old}} - \eta \frac{\partial L}{\partial w}$$

- Example:

$$w_{\text{old}} = 0.50, \quad \eta = 0.10, \quad \frac{\partial L}{\partial w} = 0.30$$
$$w_{\text{new}} = 0.50 - (0.10 \times 0.30) = 0.47$$

- Practical training usually delegates this update to an optimiser

Manual Parameter Update in TensorFlow

- A manual gradient update can be written using `assign_sub`

```
learning_rate = 0.01

for variable, gradient in zip(
    model.trainable_variables, gradients):
    variable.assign_sub(
        learning_rate * gradient)
```

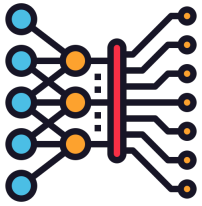
- This shows the idea behind learning
- In normal Keras training, optimisers manage these updates
- Optimisers are studied in the next lecture

Training versus Inference

- Training uses both forward propagation and backpropagation
- The forward pass produces predictions and loss
- The backward pass computes gradients for parameter updates
- Inference uses only forward propagation
- During inference, trained weights and biases remain unchanged
- Inference is generally less computationally expensive than training

Common Misunderstandings

- Forward propagation does not update weights or biases
- Backpropagation computes gradients; it does not independently choose the update rule
- The loss value and the gradient are related but not the same quantity
- Low training loss does not guarantee good performance on unseen data
- Backpropagation is used during training, not ordinary prediction
- TensorFlow automates differentiation, but the learning flow must still be understood



Deep Learning & Intelligent Systems

Summary

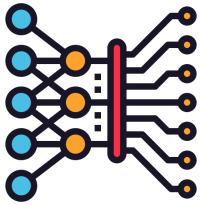
PES UNIVERSITY

- Forward propagation computes predictions from inputs
- The loss function measures prediction error
- Backpropagation computes gradients for trainable parameters
- Gradients indicate how parameter changes affect loss
- TensorFlow uses `tf.GradientTape` for automatic differentiation
- In the next lecture, optimisers use these gradients to update model parameters efficiently

Deep Learning & Intelligent Systems

Brainstorming Questions

- Why must forward propagation occur before loss can be calculated?
- Why does backpropagation begin from the output side of the network?
- What information does a gradient provide about a trainable parameter?
- Why is the chain rule important in a multilayer neural network?
- Why is backpropagation unnecessary during inference?
- What may happen if weights are updated in the same direction as the gradient?
- How does `tf.GradientTape` simplify neural-network implementation?



Deep Learning & Intelligent Systems

References

- Jon Krohn, Grant Beyleveld and Aglae Bassens, *Deep Learning Illustrated*, Pearson Education – Chapters 4 and 7
- Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press – Chapters 6 and 8
- TensorFlow 2.16 – `tf.GradientTape`
- TensorFlow 2.16 – `tf.Variable`
- TensorFlow 2.16 – `tf.reduce_mean`
- TensorFlow Guide – Introduction to gradients and automatic differentiation
- TensorFlow Keras Guide – Writing a training loop from scratch



Thank You

Niteesh K R

Assistant Professor

Department of Computer Applications

niteeshkr@pes.edu

080-26721983 Extn: 233



Deep Learning & Intelligent Systems

Deep Learning Foundations

Niteesh K R

Computer Applications

Deep Learning & Intelligent Systems

Deep Learning Foundations

Deep Learning Optimisers

Niteesh K R
Computer Applications

Why Optimisers are Needed

- A neural network begins with weights and biases that have not yet learned the required relationship
- Forward propagation produces predictions using the current parameters
- The loss function measures prediction error
- Backpropagation calculates gradients for trainable parameters
- An optimiser uses these gradients to update weights and biases
- The objective is to gradually reduce loss across training steps

Position of an Optimiser in Training

- A basic training step follows this sequence:
  - Read a batch of inputs and targets
  - Perform forward propagation
  - Calculate the loss
  - Perform backpropagation
  - Compute gradients
  - Use the optimiser to update parameters
- The updated parameters are used for the next training step

Gradient Descent Intuition

- Gradient descent is the basic idea behind many neural-network optimisers
- A gradient shows how the loss changes when a parameter changes slightly
- To reduce loss, the optimiser usually moves opposite to the gradient
- Basic update rule:

$$w_{\text{new}} = w_{\text{old}} - \eta g$$

- w is a trainable parameter
- g is the gradient of loss with respect to w
- η is the learning rate

Understanding the Learning Rate

- The learning rate controls the size of each parameter update
- A very small learning rate may make training slow
- A very large learning rate may overshoot a useful solution
- An unsuitable learning rate can cause unstable or oscillating loss
- A suitable learning rate supports steady movement toward lower loss
- Learning rate is one of the most important training hyperparameters

Gradient Descent Numerical Problem

- Consider a parameter with current value:

$$w_{\text{old}} = 2.0$$

- Suppose the gradient is:

$$g = 0.5$$

- Let the learning rate be:

$$\eta = 0.1$$

- Updated value:

$$w_{\text{new}} = 2.0 - (0.1 \times 0.5)$$

$$w_{\text{new}} = 1.95$$

- Since the gradient is positive, the parameter is decreased

Batch, Stochastic and Mini-Batch Updates

- Parameter updates can use different amounts of training data
- Batch Gradient Descent uses the complete training dataset for one update
- Stochastic Gradient Descent traditionally uses one training example per update
- Mini-batch Gradient Descent uses a small group of examples per update
- Deep Learning commonly uses mini-batches for efficient computation
- In many frameworks, the term SGD often refers to mini-batch-based training

Stochastic Gradient Descent

- SGD updates parameters using gradients from examples or mini-batches
- Basic update rule:

$$w = w - \text{learning rate} \times g$$

- SGD applies the same global learning rate to updates
- Mini-batch gradients may vary from one batch to another
- This variation can make the optimisation path fluctuate
- SGD is simple, widely studied, and computationally efficient

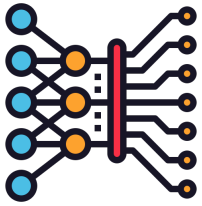
SGD with Momentum

- Standard SGD mainly uses the current gradient
- Momentum also considers the direction of previous updates
- It maintains a velocity-like value for each trainable parameter
- Momentum can accelerate movement when gradients repeatedly point in a similar direction
- It can reduce oscillation when gradients vary across dimensions
- In TensorFlow SGD, `momentum=0.0` represents standard SGD

TensorFlow SGD API

- TensorFlow provides SGD through `tf.keras.optimizers.SGD`
- Function:

  `tf.keras.optimizers.SGD(...)`
- Important parameters:
  - `learning_rate`: update step size
  - `momentum`: momentum coefficient
  - `nesterov`: enables Nesterov momentum
- Return Type:
  - SGD optimiser instance



Deep Learning & Intelligent Systems

Using SGD in a Keras Model

PES UNIVERSITY

- Create and configure an SGD optimiser

```
import tensorflow as tf

sgd = tf.keras.optimizers.SGD(
    learning_rate=0.01,
    momentum=0.9
)

model.compile(optimizer=sgd, loss="mse")
```

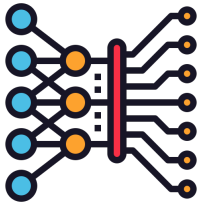
- The optimiser can be passed directly to `model.compile()`

Adam Optimiser

- Adam stands for Adaptive Moment Estimation
- Adam is a stochastic gradient-descent method with adaptive updates
- It maintains estimates based on gradients and squared gradients
- The first-moment estimate behaves similarly to momentum
- The second-moment estimate reflects recent gradient magnitude
- Adam is a widely used starting optimiser for many Deep Learning tasks

Adam Working Intuition

- Adam tracks an exponentially decaying average of previous gradients
- This first-moment estimate gives information about gradient direction
- Adam also tracks an average of squared gradients
- This second-moment estimate gives information about gradient magnitude
- Parameters with consistently large gradients may receive scaled-down updates
- Adam combines momentum-like behaviour with adaptive update scaling



Deep Learning & Intelligent Systems

TensorFlow Adam API

PES UNIVERSITY

- TensorFlow provides Adam through

  `tf.keras.optimizers.Adam`
- Function:

  `tf.keras.optimizers.Adam(...)`
- Important parameters:
  - `learning_rate`: default is commonly `0.001`
  - `beta_1`: decay rate for first-moment estimates
  - `beta_2`: decay rate for second-moment estimates
  - `epsilon`: numerical-stability constant
- Return Type: Adam optimiser instance

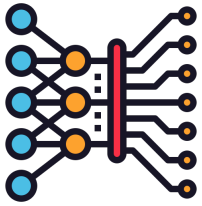
Using Adam in a Keras Model

- Create an Adam optimiser using common documented settings

```
adam = tf.keras.optimizers.Adam(
    learning_rate=0.001,
    beta_1=0.9,
    beta_2=0.999,
    epsilon=1e-7
)

model.compile(optimizer=adam, loss="mse")
```

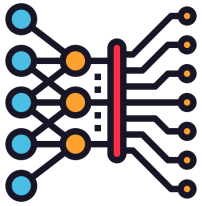
- Adam often works well as a practical first baseline



Deep Learning & Intelligent Systems

RMSProp Optimiser

- RMSProp adapts updates using recent squared gradients
- TensorFlow provides it through `tf.keras.optimizers.RMSprop`
- It maintains a discounted moving average of squared gradients
- The gradient is scaled using this moving average
- Parameters with large recent gradients receive relatively smaller effective updates
- RMSProp is useful when gradient magnitudes vary during training



Deep Learning & Intelligent Systems

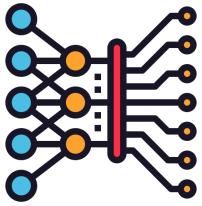
TensorFlow RMSprop API



- Function:

  `tf.keras.optimizers.RMSprop(...)`

- Important parameters:
  - `learning_rate`: update step size
  - `rho`: discounting factor for squared gradients
  - `momentum`: optional momentum coefficient
  - `epsilon`: numerical-stability constant
  - `centered`: whether to use a centered second-moment estimate
- Return Type: RMSprop optimiser instance



Deep Learning & Intelligent Systems

Using RMSprop in a Keras Model

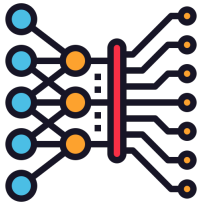
- Create a TensorFlow RMSprop optimiser

```
rmsprop = tf.keras.optimizers.RMSprop(
    learning_rate=0.001,
    rho=0.9,
    momentum=0.0,
    epsilon=1e-7,
    centered=False
)

model.compile(optimizer=rmsprop, loss="mse")
```

SGD, Adam and RMSProp Comparison

- SGD:
  - Uses a global learning rate and optional momentum
  - Simple and memory-efficient
- Adam:
  - Uses first- and second-moment estimates
  - Adapts updates for individual parameters
- RMSProp:
  - Uses a moving average of squared gradients
  - Adapts updates using recent gradient magnitude



Deep Learning & Intelligent Systems

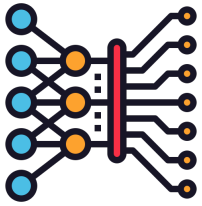
Building a Model for Optimiser Experiments

PES UNIVERSITY

- Use the same model architecture when comparing optimisers

```
def build_model():
    return tf.keras.Sequential([
        tf.keras.Input(shape=(4,)),
        tf.keras.layers.Dense(8,
activation="relu"),
        tf.keras.layers.Dense(1)
    ])
```

- Create a new model for each optimiser comparison
- This prevents one optimiser from continuing another optimiser's training



Deep Learning & Intelligent Systems

Training with a Selected Optimiser

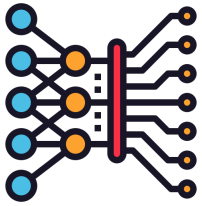
- A selected optimiser can be used in the Keras training workflow

```
model = build_model()
optimizer = tf.keras.optimizers.Adam(
    learning_rate=0.001)

model.compile(
    optimizer=optimizer,
    loss="mse")

history = model.fit(
    x_train, y_train, epochs=10, verbose=0)
```

- `fit()` repeatedly performs forward pass, loss calculation, backpropagation and updates



Deep Learning & Intelligent Systems

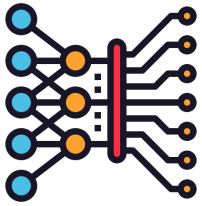
Interpreting Optimiser Experiments

PES UNIVERSITY

- Compare optimisers using the same data, architecture, loss, batch size and epochs
- Training loss shows how well the optimiser fits the training data
- Validation loss shows performance on unseen validation data
- Faster early loss reduction does not guarantee better final generalisation
- Optimiser performance can vary across datasets and architectures
- Record learning-rate values for a fair comparison

Best Practices and Common Mistakes

- Begin with Adam when a practical baseline is required
- Consider SGD with momentum when controlled and stable optimisation is desired
- Consider RMSProp when gradient magnitudes vary substantially
- Avoid comparing optimisers using different models or data splits
- Do not assume the optimiser with the lowest early training loss is always best
- Tune the learning rate before rejecting an optimiser



Deep Learning & Intelligent Systems

Summary

- Optimisers update model parameters using gradients
- Gradient descent moves parameters opposite to the gradient direction
- The learning rate controls update size
- SGD is simple and efficient
- Adam combines momentum-like behaviour with adaptive scaling
- RMSProp adapts updates using recent squared gradients
- In the next lecture, we study initialisation, normalisation and regularisation

Deep Learning & Intelligent Systems

Brainstorming Questions

- Why does an optimiser update parameters opposite to the gradient?
- What may happen when the learning rate is excessively large?
- How does momentum improve standard SGD?
- Why does Adam require more optimiser-state memory than vanilla SGD?
- How does RMSProp use squared gradients to adapt updates?
- Why must a model be reinitialised before comparing optimisers?
- Is the optimiser with the fastest training-loss reduction always the best choice?

References

- Jon Krohn, Grant Beyleveld and Aglae Bassens, *Deep Learning Illustrated*, Pearson Education – Chapter 4
- Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press – Chapter 8
- TensorFlow 2.16 – `tf.keras.optimizers.SGD`
- TensorFlow 2.16 – `tf.keras.optimizers.Adam`
- TensorFlow 2.16 – `tf.keras.optimizers.RMSprop`
- TensorFlow 2.16 – `tf.keras.Model.compile`
- TensorFlow 2.16 – `tf.keras.Model.fit`



Thank You

Niteesh K R

Assistant Professor

Department of Computer Applications

niteeshkr@pes.edu

080-26721983 Extn: 233



Deep Learning & Intelligent Systems

Deep Learning Foundations

Niteesh K R

Computer Applications

Deep Learning & Intelligent Systems

Deep Learning Foundations

Initialisation and Regularisation

Niteesh K R

Computer Applications

Why Initialisation and Regularisation Matter

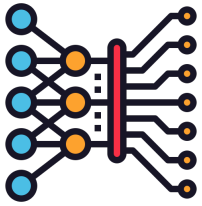
- A neural network begins training with initial values for weights and biases
- Poor initial values can slow training or make optimisation unstable
- Normalisation helps maintain stable value ranges during training
- A model may learn training data very well but perform poorly on unseen data
- Regularisation reduces the tendency to memorise training data
- Important techniques include dropout, early stopping and weight decay

Weight Initialisation

- Weight initialisation assigns starting values to trainable weights
- These values are assigned before the first forward pass
- Initialisation affects layer outputs during forward propagation
- It also affects gradients during backpropagation
- Hidden-layer weights should not all begin with identical values
- Suitable random initialisation breaks symmetry between neurons

Why Zero Weight Initialisation Fails

- Suppose all weights in a hidden layer are initialised to zero
- Neurons receive the same inputs and produce the same weighted sums
- During backpropagation, they receive identical gradients
- They continue learning identical representations
- Adding more such neurons gives no meaningful benefit
- Biases may be initialised to zero, but hidden-layer weights need suitable random values



Deep Learning & Intelligent Systems

Problems from Poor Initialisation

PES UNIVERSITY

- Very small initial weights may produce weak signals across layers
- Very large initial weights may produce excessively large activations
- Saturating activations can lead to very small gradients
- Large gradients can cause unstable parameter updates
- Signals may shrink or grow as they pass through many layers
- Variance-aware initialisers help preserve useful signal magnitudes

Glorot or Xavier Initialisation

- Glorot initialisation is also called Xavier initialisation
- It considers both the number of inputs and outputs connected to a layer
- Its goal is to maintain reasonable signal variance across layers
- It is commonly associated with sigmoid and tanh activations
- TensorFlow provides:

```
tf.keras.initializers.GlorotUniform(seed=None)
```

- Keras Dense layers use Glorot uniform as the default kernel initialiser

He Initialisation

- He initialisation is designed for ReLU-like activations
- It considers the number of inputs connected to the layer
- He normal initialisation draws values from a scaled normal distribution
- The scale helps maintain useful activation magnitudes through ReLU layers
- TensorFlow provides:

```
tf.keras.initializers.HeNormal(seed=None)
```

- It is a practical choice for Dense layers using ReLU

Glorot versus He Initialisation

- Glorot initialisation:
  - Considers layer input and output sizes
  - Commonly used with sigmoid or tanh
- He initialisation:
  - Primarily considers the number of layer inputs
  - Commonly used with ReLU-like activations
- Initialiser selection should match the activation and architecture
- Initialisation improves the starting condition but does not replace optimisation

Why Normalisation is Needed

- Neural-network inputs may use very different numerical scales
- Example: age may range from 18 to 80, while income may range into lakhs
- Large scale differences can make optimisation more difficult
- Normalisation transforms values into a more consistent range or distribution
- It can improve numerical stability and training behaviour
- Normalisation may be applied to inputs or intermediate activations

Input Normalisation Numerical Problem

- Standardisation commonly uses:

$$x_{\text{scaled}} = \frac{x-\mu}{\sigma}$$

- Suppose a feature value is:

$$x = 80, \quad \mu = 50, \quad \sigma = 10$$

- Standardised value:

$$x_{\text{scaled}} = \frac{80-50}{10} = 3$$

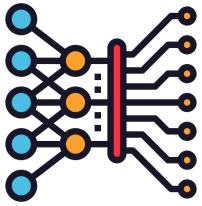
- Training-set mean and standard deviation must be reused for validation and test data

Batch Normalisation

- Batch Normalisation normalises intermediate layer inputs
- During training, it uses the mean and variance of the current batch
- It then applies a learned scale and offset
- TensorFlow provides:

```
tf.keras.layers.BatchNormalization(...)
```

- During inference, it uses stored moving statistics
- TensorFlow manages this behaviour through the layer's training mode



Deep Learning & Intelligent Systems

Using BatchNormalization in a Model

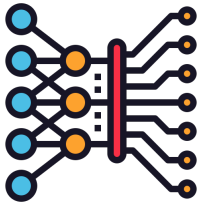
- Batch Normalisation is often placed between a Dense layer and activation

```
model = tf.keras.Sequential([
    tf.keras.Input(shape=(10,)),
    tf.keras.layers.Dense(32, use_bias=False),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.Activation("relu"),
    tf.keras.layers.Dense(1, activation="sigmoid")
])
```

- `use_bias=False` is common because Batch Normalisation includes a learned offset

Understanding Overfitting

- Overfitting occurs when a model learns the training data too specifically
- The model may memorise noise and accidental patterns
- Training loss may continue decreasing
- Validation loss may stop improving or begin increasing
- Training accuracy may become much higher than validation accuracy
- Regularisation encourages the model to learn more general relationships

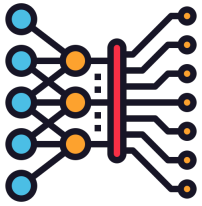


Deep Learning & Intelligent Systems

Dropout Regularisation

PES UNIVERSITY

- Dropout randomly sets a fraction of input units to zero during training
- The fraction is controlled by the `rate` parameter
- Different units may be dropped during different training steps
- This prevents the network from depending too much on particular neurons
- Dropout is active during training but not during inference
- TensorFlow provides `tf.keras.layers.Dropout`



Deep Learning & Intelligent Systems

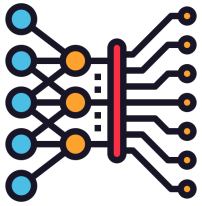
TensorFlow Dropout API

PES UNIVERSITY

- Function:

  ```
  tf.keras.layers.Dropout(rate, noise_shape=None,
  seed=None)
  ```

- Important parameters:
  - `rate`: fraction of input units set to zero
  - `noise_shape`: shape of the dropout mask
  - `seed`: optional random seed
- The rate must lie between 0 and 1
- Dropout does not remove neurons permanently



Deep Learning & Intelligent Systems

Using Dropout in a Network

PES UNIVERSITY

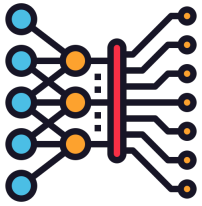
- Apply dropout after a hidden layer

```
model = tf.keras.Sequential([
    tf.keras.Input(shape=(20,)),
    tf.keras.layers.Dense(64,
        activation="relu",
        kernel_initializer="he_normal"),
    tf.keras.layers.Dropout(rate=0.3),
    tf.keras.layers.Dense(1, activation="sigmoid")
])
```

- A dropout rate of 0.3 means 30 percent of units are dropped during training

Early Stopping

- Early stopping terminates training when a monitored quantity stops improving
- Validation loss is commonly monitored
- Training may continue for a limited number of non-improving epochs
- This waiting period is controlled by `patience`
- Early stopping reduces unnecessary computation
- It can prevent training from continuing into an overfitting region



Deep Learning & Intelligent Systems

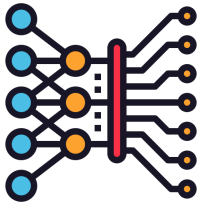
TensorFlow EarlyStopping API

PES UNIVERSITY

- TensorFlow provides:

  `tf.keras.callbacks.EarlyStopping(...)`

- Important parameters:
  - `monitor`: quantity to observe
  - `min_delta`: minimum change counted as improvement
  - `patience`: tolerated epochs without improvement
  - `mode`: `"auto"`, `"min"` or `"max"`
  - `restore_best_weights`: restores weights from the best epoch



Deep Learning & Intelligent Systems

Using EarlyStopping during Training

- Create a callback that monitors validation loss

```
early_stopping = tf.keras.callbacks.EarlyStopping(
    monitor="val_loss",
    min_delta=0.001,
    patience=5,
    mode="min",
    restore_best_weights=True)

history = model.fit(
    x_train, y_train,
    validation_data=(x_val, y_val),
    epochs=100, callbacks=[early_stopping])
```

Weight Decay and L2 Regularisation

- Weight decay discourages weights from becoming excessively large
- Smaller weights can reduce dependence on individual features or paths
- L2 regularisation adds a penalty based on squared weight values
- TensorFlow provides:

```
tf.keras.regularizers.L2(l2=0.01)
```

- The regularisation loss is added to the model's primary loss
- The regularisation strength must be selected carefully

L2 Regularisation Numerical Problem

- Suppose a layer has weights:

  $$w = [0.5,\ -0.2,\ 0.1]$$

- Let the L2 strength be:

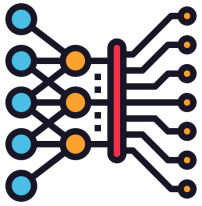
  $$\lambda = 0.01$$

- Squared-weight sum:

  $$0.5^2 + (-0.2)^2 + 0.1^2 = 0.25 + 0.04 + 0.01 = 0.30$$

- L2 penalty:

  $$0.01 \times 0.30 = 0.003$$



Deep Learning & Intelligent Systems

Using L2 Regularisation and AdamW

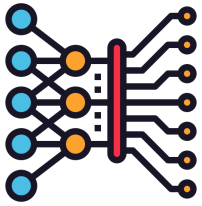
- L2 regularisation can be attached directly to a layer

```
regularizer = tf.keras.regularizers.L2(l2=0.001)

layer = tf.keras.layers.Dense(
    64, activation="relu",
    kernel_regularizer=regularizer)
```

- Decoupled weight decay is available through AdamW

```
optimizer = tf.keras.optimizers.AdamW(
    learning_rate=0.001,
    weight_decay=0.004)
```



Deep Learning & Intelligent Systems

Combined Regularised Model

- Combine initialisation, normalisation, dropout and L2 regularisation

```
model = tf.keras.Sequential([
    tf.keras.Input(shape=(20,)),
    tf.keras.layers.Dense(64, use_bias=False,
        kernel_initializer="he_normal",

kernel_regularizer=tf.keras.regularizers.L2(0.001)),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.Activation("relu"),
    tf.keras.layers.Dropout(0.3),
    tf.keras.layers.Dense(1, activation="sigmoid")
])
```

Best Practices and Common Mistakes

- Match the weight initialiser with the selected activation function
- Apply the same input-normalisation statistics to training, validation and test data
- Do not assume that more regularisation always improves performance
- Excessive dropout may cause underfitting
- Excessive weight decay may prevent useful learning
- Compare regularised and unregularised models using the same data split

Summary

- Initialisation controls the model's starting condition
- Glorot is commonly used with sigmoid or tanh activations
- He initialisation is commonly used with ReLU-like activations
- Normalisation improves numerical stability and training behaviour
- Dropout, early stopping and weight decay help reduce overfitting
- In the next class, these ideas are applied in an experiential TensorFlow neural-network workflow

Deep Learning & Intelligent Systems

Brainstorming Questions

- Why should hidden-layer weights not all be initialised to zero?
- Why is He initialisation commonly paired with ReLU?
- How does Batch Normalisation behave differently during training and inference?
- Why is dropout disabled during ordinary inference?
- How can validation loss indicate overfitting?
- What is the role of patience in early stopping?
- Can excessive regularisation cause underfitting? Justify your answer.

References

- Jon Krohn, Grant Beyleveld and Aglae Bassens, *Deep Learning Illustrated*, Pearson Education – Regularisation and improving deep networks sections
- Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press – Chapters 7 and 8
- TensorFlow 2.16 – `tf.keras.initializers.GlorotUniform`
- TensorFlow 2.16 – `tf.keras.initializers.HeNormal`
- TensorFlow 2.16 – `tf.keras.layers.BatchNormalization`
- TensorFlow 2.16 – `tf.keras.layers.Dropout`
- TensorFlow 2.16 – `tf.keras.callbacks.EarlyStopping`
- TensorFlow 2.16 – `tf.keras.regularizers.L2`
- TensorFlow 2.16 – `tf.keras.optimizers.AdamW`



Thank You

Niteesh K R

Assistant Professor

Department of Computer Applications

niteeshkr@pes.edu

080-26721983 Extn: 233



Deep Learning & Intelligent Systems

Deep Learning Foundations

Niteesh K R

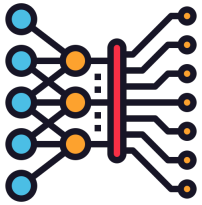
Computer Applications

Deep Learning & Intelligent Systems

Deep Learning Foundations

EL – Basic Neural Network

Niteesh K R
Computer Applications

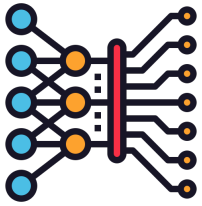


Deep Learning & Intelligent Systems

EL – Build and Train a Basic Neural Network

PES UNIVERSITY

- In this Experiential Learning activity, students will build an image-classification neural network
- The activity will be executed in a Kaggle Notebook
- The model will classify clothing images from the Fashion-MNIST dataset
- The workflow covers loading data, preprocessing, model building, training and evaluation
- TensorFlow/Keras will perform the complete training workflow
- The focus is on understanding the practical sequence, not only running code



Deep Learning & Intelligent Systems

Kaggle Notebook Setup

PES UNIVERSITY

- Open Kaggle Notebooks and create a new notebook
- Select Python as the notebook language
- A GPU accelerator is optional because Fashion-MNIST is small
- Import the required libraries:

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf

print("TensorFlow version:", tf.__version__)
```

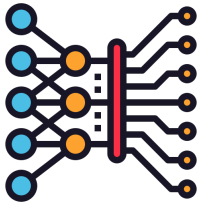
- Confirm that TensorFlow runs before continuing

Reproducibility and Random Seed

- Neural-network training includes random operations
- Examples include weight initialisation and data shuffling
- Setting a random seed improves reproducibility across repeated runs
- Add this after importing TensorFlow:

```
tf.keras.utils.set_random_seed(42)
```

- This sets Python, NumPy and TensorFlow random seeds
- Results may still vary slightly across hardware and execution environments



Deep Learning & Intelligent Systems

Loading Fashion-MNIST

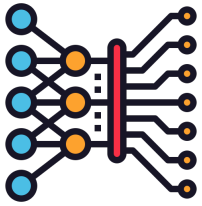
- TensorFlow provides Fashion-MNIST through `tf.keras.datasets`
- Load the dataset:

```
fashion_mnist = tf.keras.datasets.fashion_mnist

(x_train, y_train), (x_test, y_test) =
    fashion_mnist.load_data()

print(x_train.shape, y_train.shape)
print(x_test.shape, y_test.shape)
```

- The dataset is already divided into training and test sets



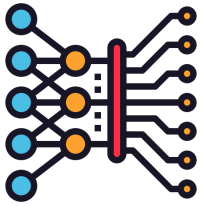
Deep Learning & Intelligent Systems

Understanding the Dataset

PES UNIVERSITY

- Fashion-MNIST contains greyscale clothing images
- Each image has size 28×28 pixels
- The training set contains 60,000 images
- The test set contains 10,000 images
- Each label is an integer from 0 to 9
- Define class names:

```
class_names = ["T-shirt/top", "Trouser",
    "Pullover", "Dress", "Coat", "Sandal",
    "Shirt", "Sneaker", "Bag", "Ankle boot"]
```



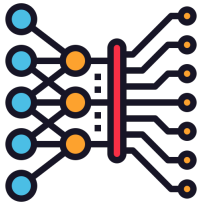
Deep Learning & Intelligent Systems

Inspecting One Training Example

- Examine the first image and its label:

```
print("Image shape:", x_train[0].shape)
print("Pixel range:",
    x_train[0].min(), x_train[0].max())
print("Label:", y_train[0])
print("Class:", class_names[y_train[0]])
```

- Pixel values are stored as integers between 0 and 255
- The numerical label identifies the clothing category



Deep Learning & Intelligent Systems

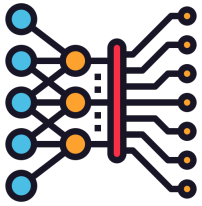
Visualising Training Images

PES UNIVERSITY

- Display sample images before building the model

```
plt.figure(figsize=(8, 8))
for i in range(9):
    plt.subplot(3, 3, i + 1)
    plt.imshow(x_train[i], cmap="gray")
    plt.title(class_names[y_train[i]])
    plt.axis("off")
plt.tight_layout()
plt.show()
```

- Visual inspection helps students connect labels with images



Deep Learning & Intelligent Systems

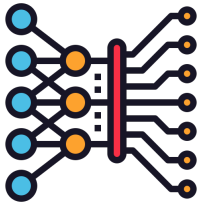
Preprocessing Pixel Values

PES UNIVERSITY

- Neural networks train better when input values are on a controlled scale
- Pixel values will be transformed from $[0, 255]$ to $[0, 1]$
- TensorFlow provides the `Rescaling` preprocessing layer
- The transformation will be included inside the model:

```
tf.keras.layers.Rescaling(1.0 / 255.0)
```

- This avoids manually changing the original image arrays
- The same preprocessing is applied during training, evaluation and prediction



Deep Learning & Intelligent Systems

Building the Neural Network

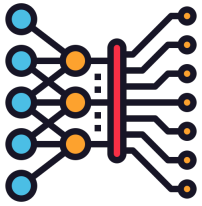
PES UNIVERSITY

- Create the model using the Keras Sequential API:

```
model = tf.keras.Sequential([
    tf.keras.Input(shape=(28, 28)),
    tf.keras.layers.Rescaling(1.0 / 255.0),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(128,
        activation="relu",
        kernel_initializer="he_normal"),
    tf.keras.layers.Dropout(0.2),
    tf.keras.layers.Dense(10,
activation="softmax")
])
```

Understanding the Model Architecture

- `Input(shape=(28, 28))` defines the shape of one image
- `Rescaling(1.0 / 255.0)` normalises pixel values
- `Flatten()` converts each image into a vector of 784 values
- `Dense(128, activation="relu")` forms the hidden layer
- `Dropout(0.2)` randomly drops 20 percent of inputs during training
- `Dense(10, activation="softmax")` produces one probability per class



Deep Learning & Intelligent Systems

Model Summary and Parameter Check

PES UNIVERSITY

- Run the following statement after creating the model:

```
model.summary()
```

- The summary shows layer types, output shapes and trainable parameters
- Classroom parameter check for the hidden Dense layer:

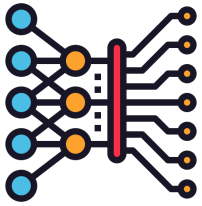
  Inputs after flattening: $28 \times 28 = 784$

  Weights: $784 \times 128 = 100352$

  Biases: 128

  Total: $100352 + 128 = 100480$

- Dense connectivity causes this layer to contain many parameters



Deep Learning & Intelligent Systems

Compiling the Neural Network

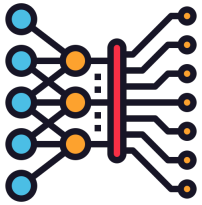
PES UNIVERSITY

- Configure the optimiser, loss function and metric:

```
model.compile(
    optimizer=tf.keras.optimizers.Adam(
        learning_rate=0.001),

loss=tf.keras.losses.SparseCategoricalCrossentropy(),
    metrics=["accuracy"]
)
```

- Sparse categorical cross-entropy is appropriate because labels are integers
- Adam updates trainable weights using gradients calculated during backpropagation



Deep Learning & Intelligent Systems

Configuring Early Stopping

- Create an EarlyStopping callback before training:

```
early_stopping = tf.keras.callbacks.EarlyStopping(
    monitor="val_loss",
    patience=2,
    mode="min",
    restore_best_weights=True
)
```

- Training stops when validation loss does not improve for two epochs
- The model restores weights from the best monitored epoch

Training the Neural Network

- Train the model using the training images and labels:

```
history = model.fit(
    x_train,
    y_train,
    validation_split=0.1,
    epochs=15,
    batch_size=64,
    callbacks=[early_stopping],
    verbose=1
)
```

- `history` stores loss and accuracy values across epochs

Plotting Training and Validation Loss

- Plot the loss recorded after every epoch:

```
plt.plot(history.history["loss"],
    label="Training Loss")
plt.plot(history.history["val_loss"],
    label="Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.show()
```

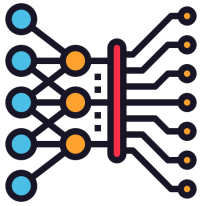
- A widening gap may indicate overfitting

Plotting Model Accuracy

- Plot training and validation accuracy:

```
plt.plot(history.history["accuracy"],
    label="Training Accuracy")
plt.plot(history.history["val_accuracy"],
    label="Validation Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.show()
```

- Accuracy shows the fraction of correctly classified examples



Deep Learning & Intelligent Systems

Evaluating the Trained Model

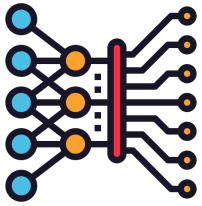
PES UNIVERSITY

- Evaluate the model using the unseen test dataset:

```
test_loss, test_accuracy = model.evaluate(
    x_test, y_test, verbose=0)

print("Test loss:", test_loss)
print("Test accuracy:", test_accuracy)
```

- Test accuracy measures the fraction of correctly classified test images
- Test loss measures disagreement between labels and predicted probabilities
- Test data must not be used for training



Deep Learning & Intelligent Systems

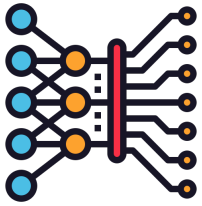
Making Predictions

- Generate predictions for the first five test images:

```
probabilities = model.predict(
    x_test[:5], verbose=0)

predicted_labels = tf.argmax(
    probabilities, axis=1).numpy()

for i in range(5):
    print("Actual:", class_names[y_test[i]])
    print("Predicted:",
        class_names[predicted_labels[i]])
```

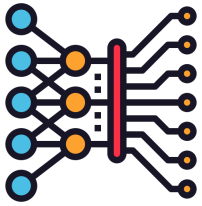


Deep Learning & Intelligent Systems

Experiential Exercises

PES UNIVERSITY

- Change hidden neurons from 128 to 64 and compare test accuracy
- Replace Adam with SGD and compare training behaviour
- Change dropout from 0.2 to 0.4 and observe validation loss
- Train without early stopping and compare the number of epochs used
- Identify five incorrectly classified test images
- Explain whether each error looks reasonable from the image



Deep Learning & Intelligent Systems

Troubleshooting Checklist

PES UNIVERSITY

- Check that each input image has shape `(28, 28)`
- Check that labels are integer class IDs from 0 to 9
- Use `SparseCategoricalCrossentropy` for integer labels
- Use `CategoricalCrossentropy` only for one-hot labels
- Confirm that `val_loss` exists before monitoring it with early stopping
- If results differ slightly, check seed, hardware and notebook restart state

References

- TensorFlow 2.16 – `tf.keras.datasets.fashion_mnist.load_data`
- TensorFlow 2.16 – `tf.keras.utils.set_random_seed`
- TensorFlow 2.16 – `tf.keras.layers.Rescaling`
- TensorFlow 2.16 – `tf.keras.layers.Flatten`
- TensorFlow 2.16 – `tf.keras.layers.Dropout`
- TensorFlow 2.16 – `tf.keras.optimizers.Adam`
- TensorFlow 2.16 – `tf.keras.losses.SparseCategoricalCrossentropy`
- TensorFlow 2.16 – `tf.keras.callbacks.EarlyStopping`
- TensorFlow 2.16 – `tf.keras.Model`



Thank You

Niteesh K R

Assistant Professor

Department of Computer Applications

niteeshkr@pes.edu

080-26721983 Extn: 233



Deep Learning & Intelligent Systems

Deep Learning Foundations

Niteesh K R

Computer Applications

Deep Learning & Intelligent Systems

Deep Learning Foundations

EL – Activation Visualisation

Niteesh K R
Computer Applications

Deep Learning & Intelligent Systems

EL – Activation Function Visualisation

- In this Experiential Learning activity, students will visualise ReLU, sigmoid and tanh
- The activity uses two environments:
  - TensorFlow Neural Network Playground
  - A Python notebook on Kaggle
- Students will observe function shape, output range and network behaviour
- The activity connects mathematical definitions with visual outputs
- No GPU accelerator is required for this experiment

Learning Outcomes and Workflow

- At the end of the activity, students should be able to:
  - Identify the output range of each activation function
  - Interpret the graphical shape of ReLU, sigmoid and tanh
  - Compare network behaviour using different activations
  - Generate activation plots using TensorFlow and Matplotlib
- Workflow:
  - Explore activations in NN Playground
  - Recreate and compare the functions in Kaggle

Opening TensorFlow Neural Network Playground

- Open:

  `https://playground.tensorflow.org/`

- NN Playground is an interactive browser-based neural-network visualisation tool
- It allows students to modify:
  - Dataset and input features
  - Hidden layers and neurons
  - Activation function
  - Learning rate and batch size
- The tool is intended for conceptual experimentation, not production modelling

Configuring the Playground Experiment

- Select the classification problem type
- Choose the circular or spiral dataset to observe a non-linear decision boundary
- Retain `x1` and `x2` as input features
- Create two hidden layers:
  - Hidden Layer 1: four neurons
  - Hidden Layer 2: four neurons
- Set the learning rate to `0.03`
- Keep regularisation as `None` for the initial comparison

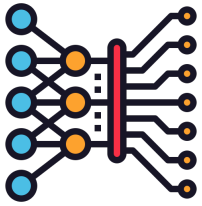
Deep Learning & Intelligent Systems

Playground Experiment – ReLU

- Select ReLU from the Activation menu
- Start training and observe the decision boundary
- ReLU returns zero for negative inputs and preserves positive inputs
- Observe hidden-neuron visualisations:
  - Some regions become inactive
  - Active regions show piecewise-linear patterns
- Record training loss and test loss after a fixed number of epochs
- Pause training before changing the activation function

Playground Experiment – Sigmoid and Tanh

- Reset or regenerate the same experiment before each comparison
- Select Sigmoid and observe the smooth transition between classes
- Sigmoid compresses outputs into the range $(0, 1)$
- Repeat the experiment using Tanh
- Tanh compresses outputs into the range $(-1, 1)$
- Compare decision-boundary shape, loss reduction and hidden-neuron patterns



Deep Learning & Intelligent Systems

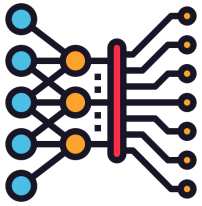
Interpreting NN Playground Colours

PES UNIVERSITY

- NN Playground uses colour to represent data, activations, weights and predictions
- Orange generally represents negative values
- Blue generally represents positive values
- In hidden layers, each neuron displays the pattern it has learned
- Connection colour indicates whether a weight is positive or negative
- Connection thickness reflects the magnitude of the weight
- The output background shows predicted regions and confidence

Recording Playground Observations

- Create an observation table with these columns:
  - Activation function
  - Epochs observed
  - Training loss
  - Test loss
  - Shape of decision boundary
  - Notable hidden-neuron patterns
- Change only the activation function during comparison
- Avoid drawing conclusions from a single run



Deep Learning & Intelligent Systems

Kaggle Notebook Setup

- Create a new Python notebook in Kaggle
- Import the required libraries:

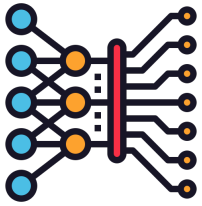
```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
```

- Display the installed TensorFlow version:

```
print(tf.__version__)
```

- Create input values from -5 to 5:

```
x = tf.linspace(-5.0, 5.0, 200)
```



Deep Learning & Intelligent Systems

Computing Activation Outputs

PES UNIVERSITY

- Apply TensorFlow activation functions to the same input tensor:

```
relu_values = tf.keras.activations.relu(x)
sigmoid_values = tf.keras.activations.sigmoid(x)
tanh_values = tf.keras.activations.tanh(x)
```

- Convert tensors to NumPy arrays for plotting:

```
x_np = x.numpy()
relu_np = relu_values.numpy()
sigmoid_np = sigmoid_values.numpy()
tanh_np = tanh_values.numpy()
```

Activation Values at Key Inputs

- Before plotting, compute outputs for $x = -2,\ 0,\ 2$
- ReLU:

  $$\text{ReLU}(-2) = 0, \quad \text{ReLU}(0) = 0, \quad \text{ReLU}(2) = 2$$

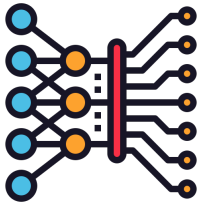
- Sigmoid:

  $$\sigma(-2) \approx 0.119, \quad \sigma(0) = 0.5, \quad \sigma(2) \approx 0.881$$

- Tanh:

  $$\tanh(-2) \approx -0.964, \quad \tanh(0) = 0, \quad \tanh(2) \approx 0.964$$

- These values should match the shapes seen in the plots



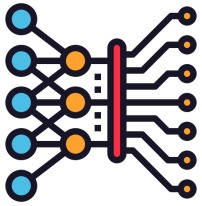
Deep Learning & Intelligent Systems

Plotting ReLU in Kaggle

- Plot the ReLU function using Matplotlib:

```
plt.figure(figsize=(8, 5))
plt.plot(x_np, relu_np, label="ReLU")
plt.axhline(0, linewidth=0.8)
plt.axvline(0, linewidth=0.8)
plt.xlabel("Input")
plt.ylabel("Output")
plt.title("ReLU Activation Function")
plt.legend()
plt.show()
```

- Observe the flat region for negative inputs and linear region for positive inputs



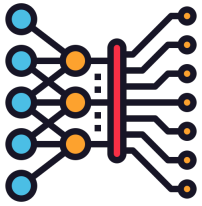
Deep Learning & Intelligent Systems

Plotting Sigmoid and Tanh

- Plot sigmoid and tanh on the same graph:

```
plt.figure(figsize=(8, 5))
plt.plot(x_np, sigmoid_np, label="Sigmoid")
plt.plot(x_np, tanh_np, label="Tanh")
plt.axhline(0, linewidth=0.8)
plt.axvline(0, linewidth=0.8)
plt.xlabel("Input")
plt.ylabel("Output")
plt.title("Sigmoid and Tanh Activations")
plt.legend()
plt.show()
```

- Compare the output ranges and zero-centred behaviour



Deep Learning & Intelligent Systems

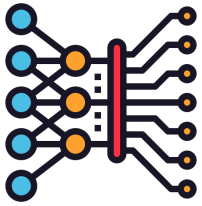
Combined Activation Comparison

- Plot all three activations together:

```
plt.figure(figsize=(9, 6))
plt.plot(x_np, relu_np, label="ReLU")
plt.plot(x_np, sigmoid_np, label="Sigmoid")
plt.plot(x_np, tanh_np, label="Tanh")
plt.xlabel("Input")
plt.ylabel("Activation Output")
plt.title("Activation Function Comparison")
plt.grid(True)
plt.legend()
plt.show()
```

Experiential Tasks

- Change the input range from $[-5, 5]$ to $[-10, 10]$
- Record the output of each activation at -2, 0 and 2
- Compare Playground behaviour for a simple dataset and a complex dataset
- Explain why sigmoid and tanh flatten for large input magnitudes
- Identify which activation is zero-centred
- Explain why ReLU is often preferred in hidden layers



Deep Learning & Intelligent Systems

Interpretation Checklist

PES UNIVERSITY

- ReLU has output range $[0, \infty)$
- Sigmoid has output range $(0, 1)$
- Tanh has output range $(-1, 1)$
- Sigmoid and tanh saturate for large positive or negative inputs
- Tanh is zero-centred, while sigmoid is not
- ReLU can create inactive regions where output is zero
- Visual behaviour should be connected back to mathematical definitions

References

- TensorFlow Neural Network Playground
- TensorFlow 2.16 – `tf.linspace`
- TensorFlow 2.16 – `tf.keras.activations.relu`
- TensorFlow 2.16 – `tf.keras.activations.sigmoid`
- TensorFlow 2.16 – `tf.keras.activations.tanh`
- Jon Krohn, Grant Beyleveld and Aglae Bassens, *Deep Learning Illustrated*, Pearson Education – Activation function discussion



Thank You

Niteesh K R

Assistant Professor

Department of Computer Applications

niteeshkr@pes.edu

080-26721983 Extn: 233